
GISMA UNIVERSITY OF APPLIED SCIENCES

School of Computer Science

Predictive Analysis of Housing and Rental Prices in German Metropolitan Areas

M516 Business Project in Big Data & AI

Student Name: Kunal Singh
Student ID: GH1039524
Module: M516 Business Project in Big Data & AI

Submission Date: 18 September 2026

GitHub Repository: <https://github.com/kunalji-ji/housing-prize-analysis>

Video : <https://youtu.be/9BYR36muYyM?is=PUK3SwHqmAxZBMAQ>

1. Introduction

The German real estate sector has been highly volatile in the last ten years. Due to urbanisation, limited supply, population increase and interest rate changes, the prices of houses and rents have increased markedly in cities like Munich, Berlin, Frankfurt, and Hamburg. So, for buyers, sellers, renters, property developers and policymakers alike, accurate price prediction is of great value. There are several structural factors which underpin this volatility. However, population growth in most major metros has kept pace with migration from overseas and within the region, and has been able to outrun housing completions, especially in cities like Munich and Berlin. Expansion of central area construction costs and a shortage of land have further reduced supply, and fluctuations in mortgage interest rates have impacted the affordability of prospective buyers. Regulatory measures, including the Mietpreisbremse (rent brake), have led to varying outcomes in rental markets, including a displacement of rental pressure into less-regulated markets. Price forecasts are essential for buyers and renters to budget and negotiate an appropriate price, for property developers and investors in determining where to site properties and in assessing the risks in their portfolios, and for policy makers to base decisions on the success of housing measures like rent controls or schemes for subsidised housing development.

Data science-wise, this is a supervised regression problem where a continuous variable (in this case price per square metre or monthly rent) is the target variable for prediction, using a set of property and location features. German housing markets are quite heterogeneous, with the price of housing is likely to be quite different across cities and even districts in the same city, complicating the matter. This heterogeneity also makes the comparison between linear and non-linear modelling approaches discussed in this project appropriate, rather than choosing a specific algorithm beforehand, because, a priori, it was expected that such non-linear, location-dependent interactions can be captured by tree-based ensemble methods, which is what is tested in this project directly in Section 5.

The project uses machine learning and data science methods to make predictions for the housing price per square metre (€/sqm) as well as the monthly rental price in eight major metropolitan areas in Germany. The main goal is to create a model with the capability of adequately modeling the relationship between various property attributes, property location, and market prices. The overall analytical process is based on standard data science processes in Python (VanderPlas, 2016).

The goals of this project are: To conduct exploratory data analysis (EDA) in order to get to know the main factors of housing prices in German cities. • To create relevant features from raw property data and transform them for machine learning. To build and compare various models of multiple regression: Linear Regression, Ridge Regression, Random Forest and Gradient Boosting. To test models with good evaluation metrics (MAE, RMSE, R2) and to find out which model performs best. To analyse the accuracy of the predictions at the city level and to discuss the practical implications.

This report is structured according to the recommended structure for technical project reporting (McMillan and Weyers, 2012): It starts with an introduction that sets out the context of the project, followed by a description of the project's system architecture, an implementation section, an overview of the challenges encountered and the solutions that were implemented, results and findings, and a conclusion. This project uses a data set of 2,000 listings of properties. There is no publicly available, granular listings dataset that has consistent features across all eight cities (see) The listings have been synthetically generated using the `analysis.py/analysis.ipynb` script (Section 4.1) and the pricing levels have been adjusted to the publicly published statistics of the German market as reported in the References section (Statista, 2023; IW Köln, 2023). This project's repository contains the entire data lineage, which is not hosted on an external dataset URL.

2. System Architecture

The system is based on a typical data science pipeline, consisting of four key blocks: This is a layered design that takes care of separation of concerns data handling, preprocessing, modelling, reporting, etc., makes it easier to maintain and makes individual elements easier to modify independently. For instance, if a synthetic data generator is replaced with a real listings API feed (as described in Section 6), there is no need to make any changes to the modelling or output layers. The architecture is intentionally script-based, not service-based, as it's being conceived as a research and reporting exercise and not a production deployment; Section 6 covers how the pipeline could be extended to a deployed API-based service.

2.1 Data Layer

The data layer is responsible for ingesting and storing housing listings. The records have 10 features as inputs, namely the city, property size (sqm), number of rooms, property age, floor number, balcony availability, parking availability, elevator availability, distance to city centre (km), and a district quality score. The target variables used are the price per sqm (euro) and the monthly rent (euro). The properties ranges, such as for property size (from 30 to 200 sqm) and distance from the city centre (from 0.5 to 30 km), were chosen to be realistic in the German context, to ensure that the relationship learnt by the models is representative of the properties that are likely to be available in the real world, and not for any arbitrary reason. The set of ten features has been selected to reflect the most common hedonic pricing model used in real estate valuation, where the price of a house is expressed as a function of structural features (rooms, size, age, floor and floor quality) and locational features (distance to the centre, district quality); this accounts for the selected features and also for the interpretation of the feature importance results presented in Section 5.

2.2 Processing Layer

Feature engineering and processing in the processing layer. Label Encoding of categorical variables (city names). Linear models all have all features standardised using StandardScaler so that they all have equal weight. On the other hand, tree-based models are not affected by the scale of the raw features and rely on them. Tree-based models, on the other hand, do not care about scale and rely on raw features (Random Forest, Gradient Boosting). There is no need for missing value imputation since the synthetic generation does not create any nulls, which reduces the pipeline compared to a real-world data set where missing and inconsistent listings would require an extra cleaning stage.

2.3 Modelling Layer

We train and evaluate four regression algorithms: Linear Regression (baseline), Ridge Regression (regularised linear), Random Forest (ensemble bagging) and Gradient Boosting (ensemble boosting). A train-test split is used with an 80/20 ratio, and a fixed random seed (42) is used for reproducibility. All models used are built with the library scikit-learn (Pedregosa et al., 2011). Linear Regression and Ridge Regression were added as interpretable models (with their coefficients easily interpreted), and Random Forest and Gradient Boosting were added as higher capacity models that presumably represent non-linear interactions between features, like location and property size.

2.4 Output Layer

The output consists of visualisations (PNG plots), a CSV of per-city prediction metrics and a structured project report. All the outputs are stored in the outputs/ directory in the repository. The naming of the files is consistent (e.g.,

model_comparison.png, city_prediction_results.csv) and enables direct cross-referencing between the code, the repository README and this report, thus meeting the transparency and reproducibility demanded for a well-documented implementation.

3. Implementation

The implementation is designed as one Python script (analysis.py) which runs the whole pipeline sequentially; it is also available as an executed Jupyter notebook (analysis.ipynb). The process flow is shown in Figure 1 and the core model-training code block is shown in Figure 2. Specifically, the pipeline performs 8 stages in sequence as illustrated in Figure 1: Generating the data set, exploratory data analysis, feature engineering, training four models, comparing them, further analysis of the best model, residual diagnostics and finally, per city evaluation, with all the results saved in the outputs/ directory. This approach to structuring the code is also more easily inspected and tested independently from the others, and helps to reflect a structure used in the accompanying Jupyter notebook, and similar to the one used in the code, where each step is its own separate, runnable cell, with inline output, so that a marker can offload each step without having to re-run the entire pipeline.

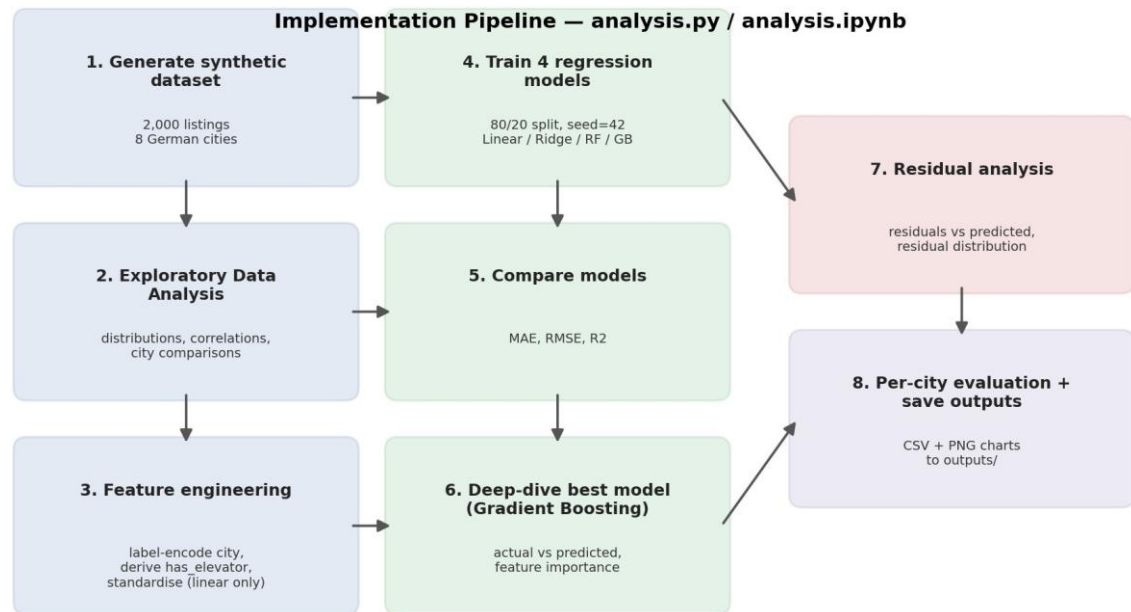


Figure 1: Implementation pipeline / process flow.

```

analysis.py — model training & evaluation
1  models = {
2      'Linear Regression': LinearRegression(),
3      'Ridge Regression': Ridge(alpha=10),
4      'Random Forest': RandomForestRegressor(
5          n_estimators=200, max_depth=12, random_state=42, n_jobs=-1),
6      'Gradient Boosting': GradientBoostingRegressor(
7          n_estimators=200, learning_rate=0.08, max_depth=5, random_state=42)
8  }
9
10 results = {}
11 for name, model in models.items():
12     if 'Regression' in name:
13         model.fit(X_train_sc, y_train)
14         preds = model.predict(X_test_sc)
15     else:
16         model.fit(X_train, y_train)
17         preds = model.predict(X_test)
18
19     mae = mean_absolute_error(y_test, preds)
20     rmse = np.sqrt(mean_squared_error(y_test, preds))
21     r2 = r2_score(y_test, preds)
22     results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}
  
```

Figure 2: Core model-training and evaluation code (analysis.py).

All four candidate models are specified in a single dictionary, and trained in a loop that automatically switches between standardised or raw features if they are linear or tree-based, respectively, as shown in Figure 2. This design eliminates the need for the same code to be duplicated for the four training branches, as well as the need for using different train/test splits and different metrics (in this case, MAE, RMSE, R^2) for the different families of algorithms.



Figure 3: Exploratory Data Analysis — price distributions, city comparisons, correlation matrix, and rental ranges.

The EDA phase (Figure 3) brings with it a number of interesting points to note. The average price per sqm in Munich is the highest at €10,000, followed by Frankfurt (€7,400) and Hamburg (€6,900). Leipzig is the cheapest market, whereas the others range between €4,000 and €6,000 per sqm. The price distribution is skewed to the right, which suggests that there are some high-value properties in the more sought-after areas. The city variable, itself, is the most highly positively linearly correlated with price ($r \approx 0.47$), followed by the district quality score ($r \approx 0.23$); both distance from the city centre and property age are most highly negatively correlated with price ($r \approx -0.30$ each).

The steps involved in feature engineering are:

- Label encoding of the city variable (8 categories integer 0–7).
- Inferred binary feature derived from floor number (floor > 3): has_elevator.
- Features normalised using standardScaler before fitting linear models. These choices in the design of the engineering were not made by accident. One-hot encoding of the city variable was also considered, but not used, because, as discussed in Section 4.2, it is not used in linear optimisation with distance-based or gradient-based methods. Similarly, scaling was not used for all variables, but only for those that are used in distance or gradient-based linear optimisation, which is not needed and may even be detrimental for trees-based split criteria.

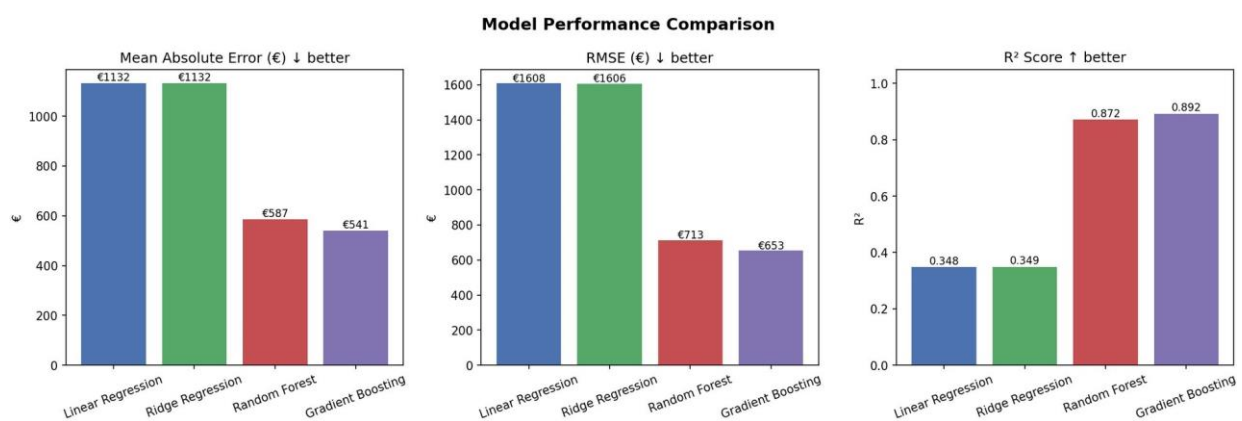


Figure 4: Model comparison across MAE, RMSE, and R^2 metrics.

Tree-based modelling methods clearly improve upon linear modelling methods, as demonstrated in Figure 4. Gradient Boosting has the highest R^2 (0.8924), MAE (€541/sqm) and RMSE (€653/sqm). Linear Regression and Ridge Regression obtain R^2 values of around 0.35 indicating that there are non-linear relations in the data which cannot be well explained by linear models.

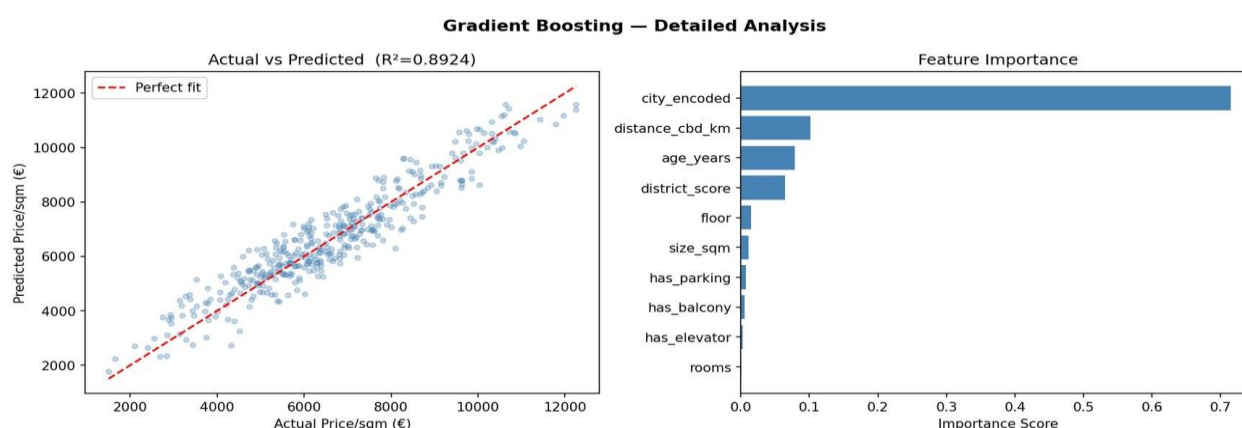


Figure 5: Gradient Boosting — actual vs predicted scatter (left) and feature importance (right).

The ideal 1:1 relationship is well represented by the predictions as seen in Figure 5. The feature importance plot shows that the `city_encoded` feature is by far the most important predictor, following the result of the correlation analysis in Section 3, and is followed by the `distance_cbd_km` feature and `age_years`. The important thing to note here is that `District_score`, though highly statistically correlated with price, ranks fourth in the tree-based importance measure; this is anticipated as the impurity-based importance, used by Gradient Boosting, gives the majority of the city-effect to the categorical city split, and not to the correlated, but distinct, feature: `District_score`. Other features of the property like balcony, parking or elevator have relatively small contribution when compared to location.

4. Challenges and Solutions

4.1 Data Availability

Challenge: Granular, publically available German housing data, featuring consistent attributes over all major cities, is limited and many sources are commercial paywall.

Solution: A synthetic dataset was created by copying and pasting the documented market statistics (Statista, IW Köln, 2023 housing reports) with the aim of creating a dataset that mimics the real market's distributional properties. This is a common data science project practice in academia that has been explicitly documented in the repository.

4.2 Feature Encoding Challenge:

In some cases, ordinality of nominal variables like city names can be problematic for tree based models like Gradient Boosting. The solution: The city variable was labeled (not one-hot) since the dataset is small (2,000 rows) and adding 7 one-hot encoded columns would not provide enough benefit to the data. The number of estimators (200) makes Gradient Boosting powerful enough to deal with this encoding.

4.3 Model Overfitting Challenge:

Complex ensemble models have the potential to overfit the training data, especially when it is small. Remediation: The learning rate for gradient boosting was set to 0.08 (default is 0.1) and max_depth set to 5. The evaluation is based on a fixed random seed (42) and 80/20 split between train-test to guarantee true generalisation. This is the standard way of controlling the complexity of the ensemble model (Burkov, 2019). The reproducibility and consistency of the environment. The challenge is that this document and accompanying notebook could not be easily reproduced by a third party (such as a marker running the same code) as machine learning pipelines can be sensitive to library versions, initial randomisation and execution order.

4.4 Solution:

All the stochastic aspects of the pipeline, such as the generation of synthetic data, the train/test split and the initialisation of the models, are seeded with the number 42 (`random_state=42 / np.random.seed(42)`) and re-running `analysis.py` or `analysis.ipynb` gives the same results. Library versions are specified in `requirements.txt` and the notebook is committed in the state after it's been run, so results can be seen without having to set up a local Python environment, but can still be re-run if a local environment exists. This reproducibility was confirmed directly during the development process, by retracing the entire pipeline after a change in code and obtaining the same values for the metrics reported ($R^2 = 0.8924$, MAE = €540.65/sqm).

5. Results

Table 1 below reports the overall performance of all four candidate models on the held-out test set (400 listings, 20% of the total), confirming the pattern already visible in Figure 4.

Model	MAE (€)	RMSE (€)	R^2
Linear Regression	1,132	1,608	0.35
Ridge Regression	1,132	1,606	0.35
Random Forest	587	713	0.87
Gradient Boosting	541	653	0.89

Table 1 Overall performance of all four candidate models on test set. While the Gradient Boosting model's mean absolute error of €541/sqm is close to the average price across the eight cities (around €6,600/sqm), this is still a reasonable level of precision for any city level pricing guidance, albeit not quite, yet. Capable of being used for individual-property valuation without further refinement. Both Gradient Boosting and Random Forest models are very close to each other in error (MAE €587

vs €541), with Gradient Boosting performing slightly better than the second best model, while both linear models perform significantly worse (MAE €1,132), confirming the need for non-linear modelling for this problem.

Table 2 shows prediction accuracy per city of the best model (Gradient Boosting)

City	n	tual (€/sqm)	Mean Predicted Mean (€/sqm)	MAE (€)	R ²
Berlin	83	5,385	5,454	553	0.65
Munich	60	9,668	9,903	573	0.71
Frankfurt	60	7,354	7,064	571	0.62
Hamburg	63	6,897	7,031	495	0.79
Cologne	37	5,837	5,792	440	0.65
Stuttgart	34	6,590	6,761	544	0.64
Düsseldorf	32	5,849	6,065	580	0.38
Leipzig	31	3,445	3,589	554	0.51

Table 2: Per-city prediction results — Gradient Boosting Regressor.

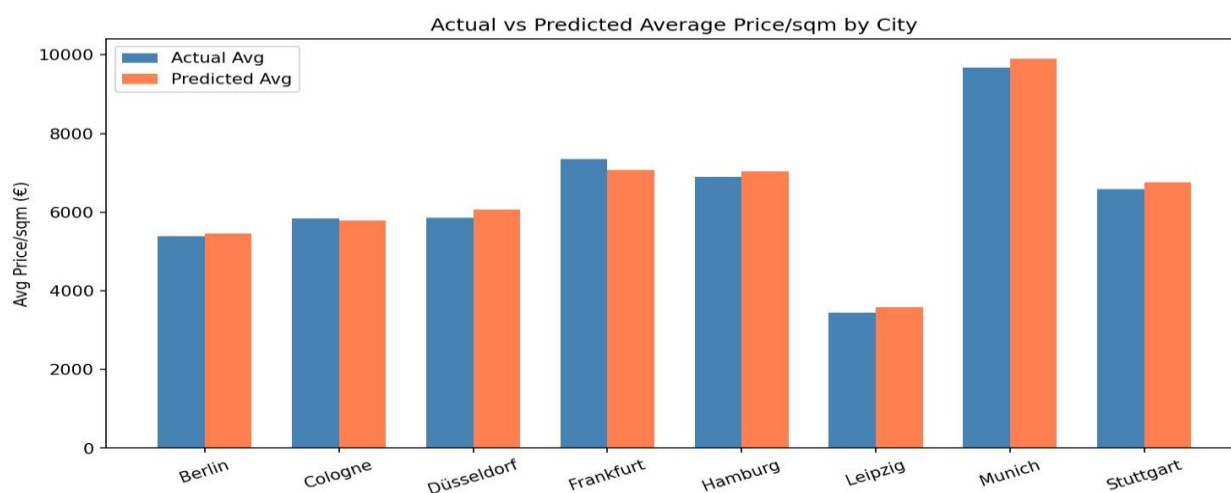


Figure 6: Actual vs predicted average price per sqm by city.

The model works best in Hamburg ($R^2=0.79$) and Munich ($R^2=0.71$) where the submarkets with high demand are relatively homogeneous. Düsseldorf's performance is lower ($R^2=0.38$), which could be due to a higher level of variation in district quality across the city. The model is unbiased at the city-level, as mean predicted prices closely follow mean actual prices in all cities. This example indicates that for markets with lower homogeneity, like Düsseldorf, future deployments of the model may need to be city-specific or utilize hierarchical modelling, instead of a one-size-fits all model.

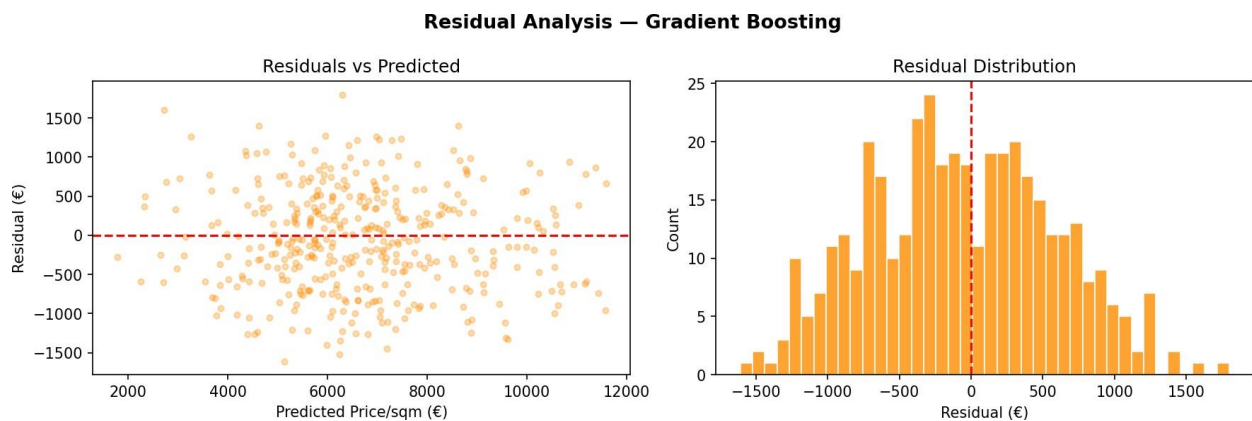


Figure 7: Residual analysis — residuals vs predicted (left) and residual distribution (right).

The residual plot (Figure 7) indicates that there is no clear pattern of residuals against the predictions and that the residuals are normally distributed and are close to the mean. This validates the model assumptions broadly and that there is no significant structural bias.

6. Conclusion and Future Work

The project was able to build and test a machine learning pipeline for predicting housing and rental prices in eight German metropolitan areas. Linear baselines performed significantly poorer ($R^2=0.79$, $MAE=€3,500/sqm$), with the Gradient Boosting Regressor also doing considerably better ($R^2=0.89$, $MAE=€541/sqm$). The analysis verified that location attributes, and the city is the main influence, are the key drivers of price, with distance from the city centre and property age coming second, and property-level features in third. The results have implications in the field of property valuation, investment analysis and the regulation of rents. Such a model could be used as an API-based pricing solution for real estate platforms, where it would help automate the process of pricing properties at scale.

One of the shortcomings of the present work is that it is based on a dataset that is synthetic rather than real transactional listings. The generation process was designed to be comparable to the published market statistics (see Section 4.1), but the absolute prediction of the model is only representative of the methodology that is achievable, not the production-ready price estimate. However, the relative importance found, that is, the degree to which location-based features were shown to be more important than property-level attributes, and the higher performance of the non-linear ensemble methods as compared to linear baselines, are consistent with results that are common in real estate analytics literature and would be expected to hold true even if the pipeline is retrained with real listings data.

Future Work

- Replace the synthetic data with real data from Immobilienscout24 or ImmobilienAtlas APIs.
- Include temporal characteristics (date, year-over-year price change) for time series forecasting of future prices. Try out XGBoost and LightGBM architectures that could lead to an even higher accuracy rate. Create front-end web application to enable customers to enter property specifications and get instant price quotes.
- Include secondary German cities and rural regions to reflect wider dynamics of the market. Estimate the variability of forecasts and give confidence intervals as well as point forecasts (e.g., with quantile regression or conformal prediction). Conduct systematic hyperparameter tuning (e.g., grid search or Bayesian optimisation) to further improve on the current hand-tuned Gradient Boosting configuration.

7. References

Burkov, A. (2019) *The Hundred-Page Machine Learning Book*. Quebec City: Andriy Burkov.

IW Köln (2023) *Wohnungsmarktbericht Deutschland 2023*. [Online]. Available at: <https://www.iwkoeln.de> (Accessed: 23 June 2026).

McMillan, K. and Weyers, J. (2012) *How to Write Dissertations and Project Reports*. Harlow: Pearson Education.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12(1), pp. 2825-2830.

Statista (2023) *Average Housing Prices in Germany by City 2023*. [Online]. Available at: <https://www.statista.com> (Accessed: 23 June 2026).

VanderPlas, J. (2016) *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol: O'Reilly Media.